

Sprawozdanie z Laboratorium

Obliczenia Naukowe - Lista 5

Wojciech Typer

4 stycznia 2026

Zadanie 1

Cel zadania

Celem zadania jest napisanie funkcji rozwiązującej układ $ax = b$ metodą eliminacji Gaussa, uwzględniając specyficzną postać macierzy A , która jest macierzą rzadką:

$$A = \begin{pmatrix} A_1 & C_1 & 0 & 0 & 0 & \dots & 0 \\ B_2 & A_2 & C_2 & 0 & 0 & \dots & 0 \\ 0 & B_3 & A_3 & C_3 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & B_{v-2} & A_{v-2} & C_{v-2} & 0 \\ 0 & \dots & 0 & 0 & B_{v-1} & A_{v-1} & C_{v-1} \\ 0 & \dots & 0 & 0 & 0 & B_v & A_v \end{pmatrix},$$

dla dwóch wariantów:

- bez wyboru elementu głównego
- z częściowym wyborem elementu głównego

Struktura danych i złożoność obliczeniowa

Macierz A jest macierzą rzadką o specyficznej strukturze blokowej, więc wystarczy tylko przechowywać jej niezerowe elementy. W programie wykorzystano strukturę słownika, co pozwala na alokację pamięci wyłącznie dla elementów niezerowych, redukując złożoność pamięciową do $O(n \cdot l)$. Złożoność czasowa procesów eliminacji i dekompozycji wynosi $O(n \cdot l^2)$, co dla małych, stałych wartości l jest złożonością liniową $O(n)$. Wynika to z faktu, że pętle algorytmu ograniczają się jedynie do niezerowego pasma macierzy, pomijając operacje na pozostałych elementach. Pozwala to na stabilne i szybkie rozwiązanie układu nawet dla bardzo dużych wartości n .

Implementacja

Idea algorytmu eliminacji Gaussa bez wyboru elementu głównego

Algorytm to „inteligentna” wersja eliminacji Gaussa, która wykorzystuje fakt, że macierz ma wartości inne niż zero tylko blisko swojej głównej przekątnej. Zamiast przeliczać całe wiersze i kolumny, program skupia się wyłącznie na wąskim pasmie o szerokości l , całkowicie ignorując ogromne obszary wypełnione zerami. Dzięki temu unikamy wykonywania milionów zbędnych działań (mnożenia przez zero), co pozwala błyskawicznie rozwiązać układ równań. Na samym końcu wynik x wyliczany jest od tyłu, również przy wykorzystaniu jedynie tych elementów, które znajdują się w pasmie.

Algorithm 1 Eliminacja Gaussa dla macierzy blokowej

```
1: procedure SOLVEGAUSS( $n, l, A, b$ )
2:   for  $k \leftarrow 1$  to  $n - 1$  do
3:      $last\_row \leftarrow \min(k + l, n)$ 
4:     for  $i \leftarrow k + 1$  to  $last\_row$  do
5:       if  $|A_{k,k}| < \epsilon$  then
6:         return error("Zero na przekątnej")
7:       end if
8:        $factor \leftarrow A_{i,k}/A_{k,k}$ 
9:        $A_{i,k} \leftarrow 0$ 
10:      for  $j \leftarrow k + 1$  to  $\min(k + l, n)$  do
11:         $A_{i,j} \leftarrow A_{i,j} - factor \cdot A_{k,j}$ 
12:      end for
13:       $b_i \leftarrow b_i - factor \cdot b_k$ 
14:    end for
15:  end for
16:   $x \leftarrow$  new vector of size  $n$ 
17:  for  $i \leftarrow n$  down to  $1$  do
18:     $sum\_val \leftarrow 0$ 
19:    for  $j \leftarrow i + 1$  to  $\min(i + l, n)$  do
20:       $sum\_val \leftarrow sum\_val + A_{i,j} \cdot x_j$ 
21:    end for
22:     $x_i \leftarrow (b_i - sum\_val)/A_{i,i}$ 
23:  end for
24:  return  $x$ 
25: end procedure
```

Idea eliminacji Gaussa z częściowym wyborem elementu głównego

Ta wersja algorytmu jest bezpieczniejsza dla wyników, ponieważ w każdym kroku szuka największej możliwej liczby w aktualnej kolumnie (tzw. elementu głównego) i zamienia wiersze tak, aby to ona znalazła się na przekątnej. Dzięki temu unikamy dzielenia przez bardzo małe liczby, co drastycznie poprawia dokładność obliczeń i chroni przed błędami zaokrągleń.

Algorithm 2 Eliminacja Gaussa z częściowym wyborem elementu głównego

```
1: procedure SOLVEGAUSSPIVOT( $n, l, A, b$ )
2:    $p \leftarrow [1, 2, \dots, n]$  ▷ Inicjalizacja wektora permutacji
3:   for  $k \leftarrow 1$  to  $n - 1$  do
4:      $max\_val \leftarrow 0.0$ 
5:      $pivot\_row \leftarrow k$  ▷ Wybór elementu głównego w obrębie pasma  $l$ 
6:     for  $i \leftarrow k$  to  $\min(k + l, n)$  do
7:       if  $|A_{p_i, k}| > max\_val$  then
8:          $max\_val \leftarrow |A_{p_i, k}|$ 
9:          $pivot\_row \leftarrow i$ 
10:      end if
11:    end for
12:     $swap(p_k, p_{pivot\_row})$  ▷ Zamiana wierszy w wektorze permutacji
13:    for  $i \leftarrow k + 1$  to  $\min(k + l, n)$  do
14:       $factor \leftarrow A_{p_i, k} / A_{p_k, k}$ 
15:       $A_{p_i, k} \leftarrow 0.0$  ▷ Aktualizacja wiersza - pasmo może wzrosnąć do  $2l$ 
16:      for  $j \leftarrow k + 1$  to  $\min(k + 2l, n)$  do
17:         $A_{p_i, j} \leftarrow A_{p_i, j} - factor \cdot A_{p_k, j}$ 
18:      end for
19:       $b_{p_i} \leftarrow b_{p_i} - factor \cdot b_{p_k}$ 
20:    end for
21:  end for
22:   $x \leftarrow$  nowy wektor rozmiaru  $n$  ▷ Podstawienie wsteczne z uwzględnieniem permutacji i pasma  $2l$ 
23:  for  $i \leftarrow n$  down to  $1$  do
24:     $sum\_val \leftarrow 0.0$ 
25:    for  $j \leftarrow i + 1$  to  $\min(i + 2l, n)$  do
26:       $sum\_val \leftarrow sum\_val + A_{p_i, j} \cdot x_j$ 
27:    end for
28:     $x_i \leftarrow (b_{p_i} - sum\_val) / A_{p_i, i}$ 
29:  end for
30:  return  $x$ 
31: end procedure
```
